

NLP任务和基本问题

目录

CONTENTS

NLP任务

基本问题

评价指标

目录

□ NLP任务

□ 语言模型

□ 基础任务

- 中文分词

- 字词切分

- 词性标注

- 句法分析

- 语义分析

□ 应用任务

- 信息提取

- 情感分析

- 问答系统

- 机器翻译

- 对话系统

□ 基本问题

- 文本分类

- 结构预测

- 序列到序列

NLP任务-语言模型

- **语言模型**(Language Model , LM) : 是描述**自然语言概率分布**的模型
 - 统计语言模型
 - 一个非常基础和重要的NLP任务
 - 利用LM , 可计算**一个词序列或一句话**的概率
 - 可在**给定上文**的条件下对**接下来可能出现的词**进行概率分布估计
- **预训练语言模型**: 语言模型是一项天然的预训练任务 , 在基于预训练模型的NLP方法中起到非常重要的作用
- **经典的N元语言模型**(N-gram Language Model)
- **现代神经网络语言模型**(Neural Network Language Model)(第5章)

NLP任务-语言模型

□ 语言模型 (Language Model , LM)

□ 描述一段自然语言的概率或给定上文时下一个词出现的概率

□ $P(w_1, \dots w_l) , P(w_{l+1}|w_1, \dots w_l)$

□ 以上两种定义等价 (链式法则)

$$\begin{aligned} P(w_1 w_2 \cdots w_l) &= P(w_1) P(w_2|w_1) P(w_3|w_1 w_2) \cdots P(w_l|w_1 w_2 \cdots w_{l-1}) \\ &= \prod_{i=1}^l P(w_i|w_{1:i-1}) \end{aligned}$$

□ 广泛应用于多种自然语言处理任务

□ 机器翻译 (词排序)

□ $P(\text{the cat is small}) > P(\text{small the is cat})$

□ 语音识别 (词选择)

□ $P(\text{there are four cats}) > P(\text{there are for cats})$

NLP任务-语言模型-N元语言模型

□ **语言模型的基本任务**：给定词序列 $w_1 w_2 \cdots w_{t-1}$ ，估计下一时刻t可能出现的词 w_t 的条件概率 $P(w_t | w_1 w_2 \cdots w_{t-1})$

□ $w_1 w_2 \cdots w_{t-1} : w_t$ 的历史

□ **例如**：对于历史“我喜欢”，下一个词为“读书”的概率，即： $P(\text{读书} | \text{我喜欢})$

□ 在给定一个**语料库**时，该条件概率可以理解为当语料中出现“我喜欢”时，有多少次下一个词为“读书”，通过**最大似然估计**进行计算：

$$P(\text{读书} | \text{我喜欢}) = \frac{C(\text{我喜欢 读书})}{C(\text{我喜欢})} \quad (2-9)$$

□ $C(\cdot)$ ：词序列在语料库中出现的**次数(频次)**

NLP任务-语言模型-N元语言模型

□ 通过以上条件概率，可进一步计算一个句子出现的概率，即相应单词序列的联合概率 $P(w_1 w_2 \cdots w_l)$ ， l 为序列的长度

□ 利用链式法则对该式进行分解，从而将其转化为条件概率的计算问题，即：

$$\begin{aligned} P(w_1 w_2 \cdots w_l) &= P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) \cdots P(w_l | w_1 w_2 \cdots w_{l-1}) \\ &= \prod_{i=1}^l P(w_i | w_{1:i-1}) \end{aligned} \quad (2-10)$$

□ $w_{i:j}$ 表示由位置 i 到 j 的子串 $w_i w_{i+1} \cdots w_j$

□ 例：训练语料库，只包含以下两个句子（已分词）：

□ 句子1：小明 喜欢 吃 苹果

□ 句子2：小红 喜欢 喝 牛奶

□ 评估新句子：小明 喜欢 喝 可乐

$$P(\text{小明喜欢喝可乐}) = P(\text{小明}) \cdot P(\text{喜欢} | \text{小明}) \cdot P(\text{喝} | \text{小明喜欢}) \cdot P(\text{可乐} | \text{小明喜欢喝})$$

NLP任务-语言模型-N元语言模型

□ **问题(零概率)** : $P(w_i | w_{1:i-1}) = 0$, 句子长度增加 , $w_{1:i-1}$ 出现的次数会越来越少 , 甚至从未出现过

□ **例** : 训练语料库 , 只包含以下两个句子 (已分词) :

□ 句子1 : 小明 喜欢 吃 苹果

□ 句子2 : 小红 喜欢 喝 牛奶

□ 评估新句子 : 小明 喜欢 喝 可乐

$P(\text{小明喜欢喝可乐}) = P(\text{小明}) \cdot P(\text{喜欢} | \text{小明}) \cdot P(\text{喝} | \text{小明喜欢}) \cdot P(\text{可乐} | \text{小明喜欢喝})$

NLP任务-语言模型-N元语言模型

□ **问题(零概率)** : $P(w_i | w_{1:i-1}) = 0$, 句子长度增加, $w_{1:i-1}$ 出现的次数会越来越少, 甚至从未出现过

□ **马尔可夫假设**(Markov Assumption): “下一个词出现的概率只依赖于它前面 $n-1$ 个词” , 即 :

$$P(w_t | w_1 w_2 \cdots w_{t-1}) \approx P(w_t | w_{t-(n-1):t-1}) \quad (2-11)$$

□ **N元语法或N元文法(N-gram)**模型:满足**马尔可夫假设**假设的模型

□ **一元语法** : 当 $n=1$ 时, 下一个词的出现独立于其历史, 通常记作unigram

□ **二元语法** : 当 $n=2$ 时, 下一个词只依赖于前1个词, 记作bigram, 一阶马尔可夫链(Markov Chain)

□ **三元语法**: 假设($n=3$)、二阶马尔可夫假设, 记作trigram

NLP任务-语言模型-N元语言模型

- n 的取值越大，考虑的历史越完整。在unigram模型中，由于词与词之间相互独立，因此它是与语序无关的
- 以bigram模型为例，式 (2-10) 可转换为：

$$P(w_1 w_2 \cdots w_l) = \prod_{i=1}^l P(w_i | w_{i-1}) \quad (2-12)$$

- 为了使 $P(w_i | w_{i-1})$ 对于 $i=1$ 有意义
 - 在句子的开头增加一个句首标记 < BOS > (Begin Of Sentence), 并设 $w_0 = < BOS >$
 - 在句子的结尾增加一个句尾标记 < EOS > (End Of Sentence), 设 $w_{l+1} = < EOS >$

NLP任务-语言模型-平滑

$$\begin{aligned} P(w_1 w_2 \cdots w_l) &= P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) \cdots P(w_l | w_1 w_2 \cdots w_{l-1}) \\ &= \prod_{i=1}^l P(w_i | w_{1:i-1}) \end{aligned} \quad (2-10)$$

□ “零概率” 问题：

□ 马尔可夫假设降低了句子概率为0的可能性

□ 当n比较大

□ 测试句子中含有未登录词(Out-Of-Vocabulary, OOV)

□ 由于数据的稀疏性，训练数据很难覆盖测试数据中所有可能出现的N-gram，但这并不意味着这些N-gram出现的概率为0

□ 解决办法：平滑(Smoothing) 技术调整概率估计的结果

□ 折扣法：最基本，最简单的平滑算法

NLP任务-语言模型-平滑

□ **折扣法**(Discounting)平滑的基本思想: “**损有余而补不足**”

□ 从**频繁**出现的N-gram中匀出一部分概率并分配给**低频次**(含零频次)的N-gram, 从而使得**整体概率分布趋于均匀**

□ **加1平滑**(Add-one Discounting): 典型的折扣法, 也称为**拉普拉斯平滑**(Laplace Smoothing), 假设所有N-gram的**频次比实际出现的频次多一次**

□ 例如: **unigram模型**平滑之后的概率, 可由以下公式计算:

$$P(w_i) = \frac{C(w_i) + 1}{\sum_w (C(w) + 1)} = \frac{C(w_i) + 1}{N + |\mathbb{V}|} \quad (2-13)$$

□ $|\mathbb{V}|$: 词表大小

□ 所有**未登录词**可映射为一个区别于其他已知词汇的独立标记, 如 < UNK >

NLP任务-语言模型-平滑

□ 对于bigram模型，则有：

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{\sum_w (C(w_{i-1}w) + 1)} = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |\mathbb{V}|} \quad (2-14)$$

□ 实际应用中，尤其当训练数据较小时，加1平滑将对低频次或零频次事件给出过高的概率估计

□ 一种自然的扩展是加 δ 平滑。在加 δ 平滑中，假设所有事件的频次比实际出现的频次多 δ 次，其中 $0 \leq \delta \leq 1$

NLP任务-语言模型-平滑

□ 以bigram语言模型为例，使用加 δ 平滑之后的条件概率为：

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + \delta}{\sum_w (C(w_{i-1}w) + \delta)} = \frac{C(w_{i-1}w_i) + \delta}{C(w_{i-1}) + \delta|\mathbb{V}|} \quad (2-15)$$

- 关于超参数 δ 的取值，需要用到开发集数据
- 根据开发集上的困惑度对不同 δ 取值下的语言模型进行评价，最终将最优的 δ 用于测试集

□ N元语言模型缺陷：

- ① 无法对长度超过N 的长距离词语依赖关系进行建模，因引入马尔可夫假设
- ② 如果将 N 扩大
 - ① 带来更严重的数据稀疏问题
 - ② 急剧增加模型的参数量(N-gram数目)
 - ③ 存储和计算都带来极大的挑战

□ 5.1节将要介绍的神经网络语言模型可以较好地解决N元语言模型的这些缺陷。

N-gram例子

□语料库

□只有一个句子："the cat sat on the mat"

□分词：["the", "cat", "sat", "on", "the", "mat"]

□1-gram (unigram)：每个单词单独出现，统计频次：

□the: 2

□cat: 1

□sat: 1

□on: 1

□mat: 1

N-gram例子

□语料库

□只有一个句子："the cat sat on the mat"

□分词：["the", "cat", "sat", "on", "the", "mat"]

□2-gram (bigram)：连续两个词组成的对，统计频次：

□(the, cat): 1

□(cat, sat): 1

□(sat, on): 1

□(on, the): 1

□(the, mat): 1

N-gram例子

□语料库

□只有一个句子："the cat sat on the mat"

□分词：["the", "cat", "sat", "on", "the", "mat"]

□3-gram (trigram)

□连续三个词组成的序列，统计频次：

□(the, cat, sat): 1

□(cat, sat, on): 1

□(sat, on, the): 1

□(on, the, mat): 1

注意：n-gram通常不跨句子边界

语言模型性能评价

- **外部任务评价**: 将语言模型应用于**具体的外部任务**(如机器翻译), 并根据该任务上**指标的高低**对语言模型进行评价
 - **最接近实际应用需求**的一种评价方法
 - **计算代价较高**, 实现的**难度也较大**
- **内部评价**: 目前最为常用的是基于**困惑度**(Perplexity , PPL)的“方式”

NLP任务-语言模型-语言模型性能评价

困惑度内部评价

- a) 将数据划分为不相交的两个集合：
 - a) 训练集 D^{train} ：估计语言模型的参数
 - b) 测试集 D^{test} ：性能测试
- b) 由模型计算出的测试集的概率 $P(\mathbb{D}^{\text{test}})$ ，其反映了模型在测试集上的泛化能力
- c) 假设测试集 $\mathbb{D}^{\text{test}} = w_1 w_2 \cdots w_N$ (每个句子的开始和结束分布增加 $\langle \text{BOS} \rangle$ 与 $\langle \text{EOS} \rangle$ 标记)，那么测试集的概率为：

$$\begin{aligned} P(\mathbb{D}^{\text{test}}) &= P(w_1 w_2 \cdots w_N) \\ &= \prod_{i=1}^N P(w_i | w_{1:i-1}) \end{aligned} \tag{2-16}$$

NLP任务-语言模型-语言模型性能评价

- d) 惑度则为模型分配给测试集中每一个词的概率的几何平均值的倒数：

$$\text{PPL}(\mathbb{D}^{\text{test}}) = \left(\prod_{i=1}^N P(w_i | w_{1:i-1}) \right)^{-\frac{1}{N}}$$

- e) 对于bigram模型而言：
- $$\text{PPL}(\mathbb{D}^{\text{test}}) = \left(\prod_{i=1}^N P(w_i | w_{i-1}) \right)^{-\frac{1}{N}}$$

- f) 在实际计算过程中，考虑到多个概率的连乘可能带来浮点数下溢的问题，通常需要上式转化为对数和的形式：

$$\text{PPL}(\mathbb{D}^{\text{test}}) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_{i-1})}$$

NLP任务-语言模型-语言模型性能评价

困惑度：

- PPL越小，对应句子出现的概率就越高
- 可以认为是average branch factor(平均分支系数)
 - 即预测下一个词时可以有多少种选择
- 模型的PPL下降到90，可直观地理解为，在模型生成一句话时下一个词有90个合理选择，可选词数越少，大致认为模型越准确。这样也能解释，为什么PPL越小，模型越好。

NLP任务-语言模型-语言模型性能评价

- 困惑度**越小**，意味着单词序列的概率越大，也意味着**模型能够更好地解释测试集中的数据**
- 需要注意的是，困惑度越低的语言模型并**不总是**能在外部任务上取得**更好的性能指标**，但是两者之间通常呈现出一定的**正相关性**
- 困惑度可以作为一种**快速评价**语言模型性能的指标，而在将其应用于**下游任务**时，仍然需要根据其在**具体任务上的表现**进行评价

□ 语言模型

- N 元语言模型

- 平滑

- 语言模型评价

目录

□ NLP任务

□ 语言模型

□ 基础任务

- 中文分词

- 字词切分

- 词性标注

- 句法分析

- 语义分析

□ 应用任务

- 信息提取

- 情感分析

- 问答系统

- 机器翻译

- 对话系统

□ 基本问题

- 文本分类

- 结构预测

- 序列到序列

NLP任务-基础任务-中文分词

- **词(Word)**:最小的能独立使用的音义结合体，是能够独立运用并能够表达语义或语用内容的最基本单元
- **英语**为代表的**印欧语系**: 词之间用分隔符(空格等)区分
- **汉语**为代表的**汉藏语系**和**阿拉伯语**为代表的**闪-含语系**:不包含明显的词之间的分隔符
- 为了进行后续的自然语言处理，通常需要首先对不含分隔符的语言进行**分词(Word Segmentation)**操作
- 本节以中文分词为例，介绍词的**切分**问题和**最简单的分词**算法
- **中文分词**：将中文字序列切分成一个个单独的词
 - 输入：“我喜欢读书”
 - 输出：“我 喜欢 读书”

NLP任务-基础任务-中文分词

□分词：严守一把手机关了 严守一/ 把/ 手机/ 关/ 了

□分词的歧义

□严守一/ 把/ 手机/ 关/ 了

□严守/ 一把手/ 机关/ 了

□严守/ 一把/ 手机/ 关/ 了

□严守一/ 把手/ 机关/ 了

□.....

□正向最大匹配(Forward Maximum Matching , FMM)分词算法：最简单的分词算法，即从前向后扫描句子中的字符串，尽量找到词典中较长的单词作为分词的结果。具体代码如下：

NLP任务-基础任务-中文分词

```
def fmm_word_seg(sentence, lexicon, max_len):  
    """  
    sentence: 待分词的句子  
    lexicon: 词典（所有单词集合）  
    max_len: 词典中最长单词长度  
    """  
    begin = 0  
    end = min(begin + max_len, len(sentence))  
    words = []  
    while begin < end:  
        word = sentence[begin:end]  
        if word in lexicon or end - begin == 1:  
            words.append(word)  
            begin = end  
            end = min(begin + max_len, len(sentence))  
        else:  
            end -= 1  
    return words
```

NLP任务-基础任务-中文分词

□通过下面的代码加载词典并调用正向最大匹配分词算法：

```
def load_dict():
    f = open("lexicon.txt") # 词典文件，每行存储一个单词
    lexicon = set()
    max_len = 0
    for line in f:
        word = line.strip()
        lexicon.add(word)
        if len(word) > max_len:
            max_len = len(word)
    f.close()

    return lexicon, max_len

lexicon, max_len = load_dict()
words = fmm_word_seg(input("请输入句子: "), lexicon, max_len)

for word in words:
    print(word,)
```

NLP任务-基础任务-中文分词

❑正向最大匹配分词算法存在的明显缺点(分词错误)：

❑切分歧义：倾向于切分出较长的词，导致错误的切分结果

❑如“研究生命的起源”，分为“研究生 命 的 起源”

❑分词错误，则会影响对句子的语义理解

❑中文词的定义也不明确：如“哈尔滨市”可以是一个词，“哈尔滨”是一个词，“市”是一个词

❑存在多种中文分词的规范，根据不同规范标注了不同的数据集

❑未登录词问题：有一些词并没有收录在词典中

❑如新词、命名实体、领域相关词和拼写错误词等

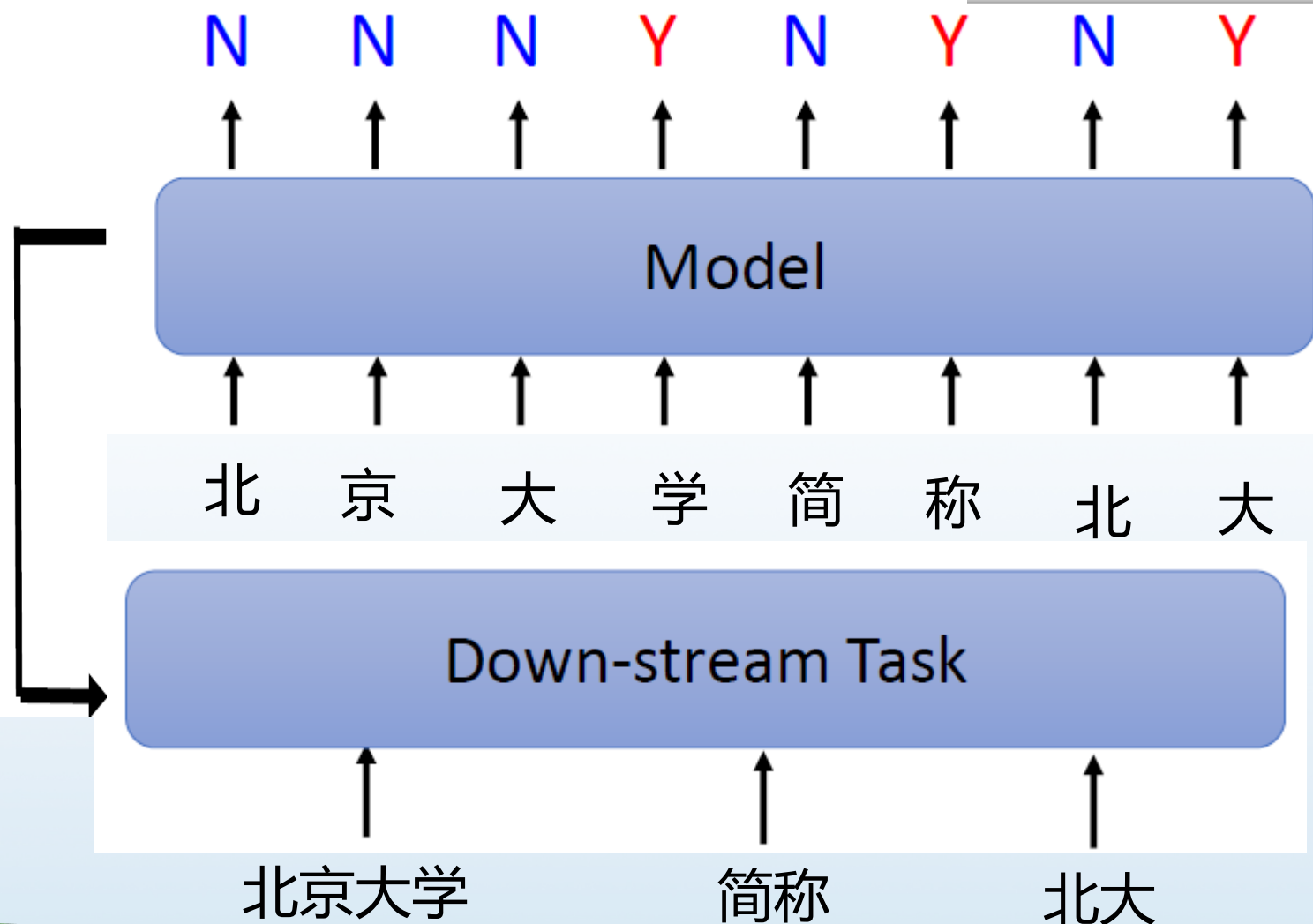
❑语言的动态性，新词语的出现可谓是层出不穷

❑相比切分歧义问题，真实应用环境中，由未登录词问题引起的分词错误比例更高。

❑分词任务本身是一项富有挑战的NLP基础任务

NLP任务-基础任务-中文分词

Input: sequence
Output: class for each token



NLP任务-基础任务-子词切分(Subword)

- 以英语为代表的印欧语系语言，是否需要进行分词？
- 这些语言词形变化复杂
 - 如：*computer*、*computers*、*computing*等
- 仅用空格切分的问题
 - 数据稀疏
 - 词表过大，降低处理速度
- 子词切分
 - 将一个单词切分为若干连续的片段（子词）
 - 方法众多，基本原理相似
 - 使用尽量长且频次高的子词对单词进行切分

NLP任务-基础任务-子词切分(Subword)

❑ **传统**的处理方法: 根据语言学规则, 引入**词形还原**或者**词干提取**等任务, 提取出单词的**词根**

❑ **词形还原**(Lemmatization): 将变形的词语转换为**原形**

❑ 如将 “computing” 还原为 “compute” ;

❑ **词干提取**(Stemming): 将前缀、后缀等去掉, 保留**词干**(Stem)

❑ 如 “computing” 的词干为 “comput”, 可见, 词干提取的结果可能不是一个完整的单词

❑ **效果**: 一定程度上克服数据稀疏问题

❑ **问题**:

❑ 需要人工撰写大量的规则

❑ 既不容易扩展到新的领域, 也不容易扩展到新的语言上

❑ **基于统计的无监督子词(Subword)切分**任务应运而生, 并在现代的预训练模型中使用

NLP任务-基础任务-子词切分(Subword)

□子词切分

- 将一个单词切分为若干连续的**片段**（子词）

- 方法众多，**基本原理相似**

 - 使用**尽量长**且**频次高**的子词对单词进行切分

□字节对编码(Byte Pair Encoding , BPE)算法

- 何构造子词词表

- 字词切分(句子进行编码)

- 编码后的句子进行解码

NLP任务-基础任务-子词切分(Subword)

Algorithm 1: BPE 中子词词表构造算法

Input: 大规模生文本语料库；期望的子词词表大小 L

Output: 子词词表

将语料库中每个单词切分成字符作为子词；

用切分的子词构成初始子词词表；

while 子词词表小于等于 L **do**

 在语料库中统计单词内相邻子词对的频次；

 选取频次最高的子词对，合并成新的子词；

 将新的子词加入子词词表；

 将语料库中不再存在的子词从子词词表中删除；

end

子词词表构造示例

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

初始子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 's', 't', 'i', 'd'}

合并子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 't', 'i', 'd', 'es'}

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

合并子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 'i', 'd', 'est'}

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

NLP任务-基础任务-子词切分(Subword)

□BPE子词切分算法(贪心的方法)

- a) 将子词词表按照子词的长度由大到小进行排序
- b) 从前向后遍历子词词表，依次判断一个子词是否为单词的子串
- c) 如果是则将该单词进行切分，然后继续向后遍历子词词表
- d) 如果子词词表全部遍历结束，单词中仍然有子串没有被切分，那么这些子串一定为低频串，则使用统一的标记，如'<UNK>'进行替换

□例：

- 切分句子: ['the </w>', 'highest </w>', 'mountain </w>']
- 词表: ['errrr </w>', 'tain </w>', 'moun', 'est </w>', 'high', 'the </w>', 'a </w>']
- 词切分结果: ['the </w>', 'high', 'est </w>', 'moun', 'tain </w>']

NLP任务-基础任务-子词切分(Subword)

□例：

- 切分句子: ['the < /w > ', 'highest < /w > ', 'mountain < /w > ']
- 词表: ['errrr < /w > ', 'tain < /w > ', 'moun', 'est < /w > ', 'high', 'the < /w > ', 'a < /w > ']
- 词切分结果: ['the < /w > ', 'high', 'est < /w > ', 'moun', 'tain < /w > ']

□解码:还原成原始的句子

- 只要将全部子词进行拼接，然后将结尾字符替换为空格，就恰好为原始的句子了
 - 单词结尾字符' < /w > '便发挥作用

NLP任务-基础任务-子词切分(Subword)

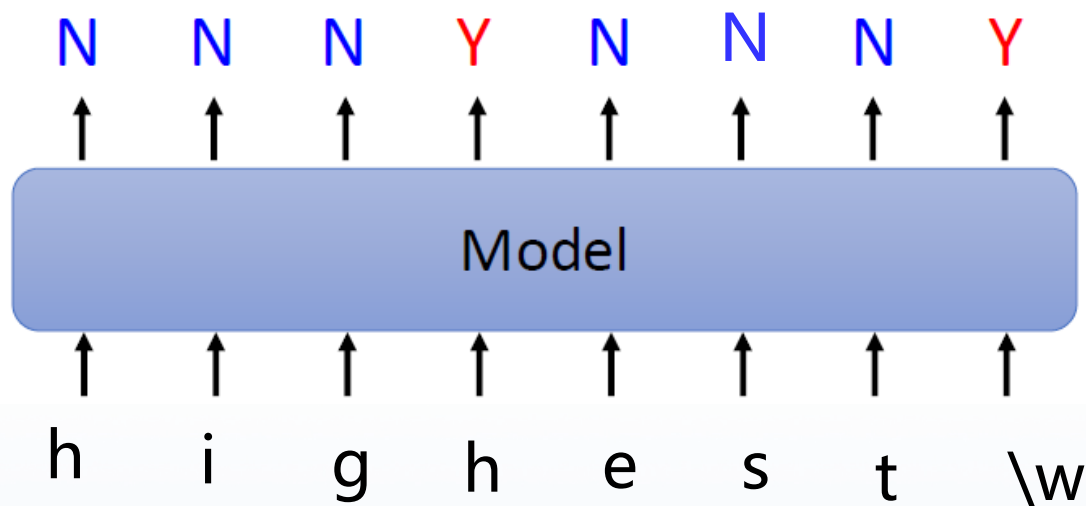
□ BPE子词切分算法问题：

- BPE算法中的编码步骤需要遍历整个词表，是一个非常耗时的过程
- 通过缓存技术加快编码的速度
 - 即将常见单词对应的编码结果事先存储下来，然后编码时通过查表的方式快速获得编码的结果
 - 对于查不到的单词再实际执行编码算法
 - 高频词能够覆盖语言中的大部分单词
 - 实际执行编码算法的次数并不多，可以极大地提高编码过程的速度

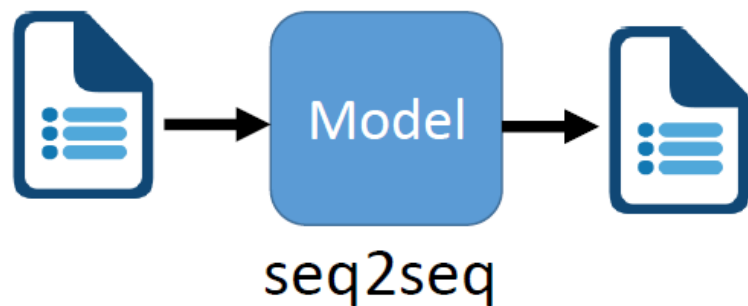
□ 更多子词切分算法

- WordPiece
- Unigram Language Model (ULM)
- SentencePiece
 - <https://github.com/google/sentencepiece>

NLP任务-基础任务-中文分词



Input: sequence
Output: class for each token



Input: sequence
Output: sequence

NLP任务-基础任务-词性标注

□**词性**：词语在句子中扮演的语法角色，词类(Part-Of-Speech, POS), 如:

□**名词**: 表示抽象或具体事物名字(如 “计算机”)

□**动词**: 表示动作(如 “打”)、状态(如 “存在”)的词

□**词性作用**：可为**句法分析**、**语义理解**等提供帮助

□**词性标注**(POS Tagging)：给定一个句子，输出句子中每个词相应的词性

□**例如**：输入 `他 喜欢 下 象棋 。`

□**输出**： `他/PN 喜欢/VV 下/VV 象棋/NN 。/PU`

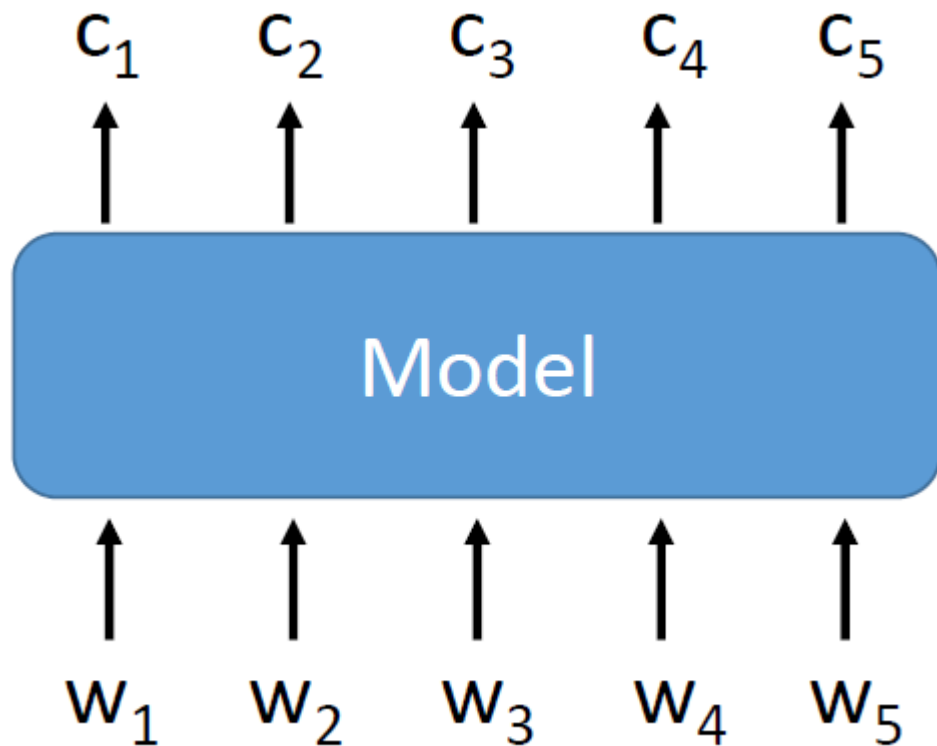
□PN、VV、NN和PU分别代表**代词**、**动词**、**名词**和**标点符号**

□**词性标注主要难点**：**歧义性**，词在不同的上下文中可能有不同的词性

□例如，上例中的“下”，既可以表示动词，也可以表示方位词

□需要结合上下文确定词的具体词性

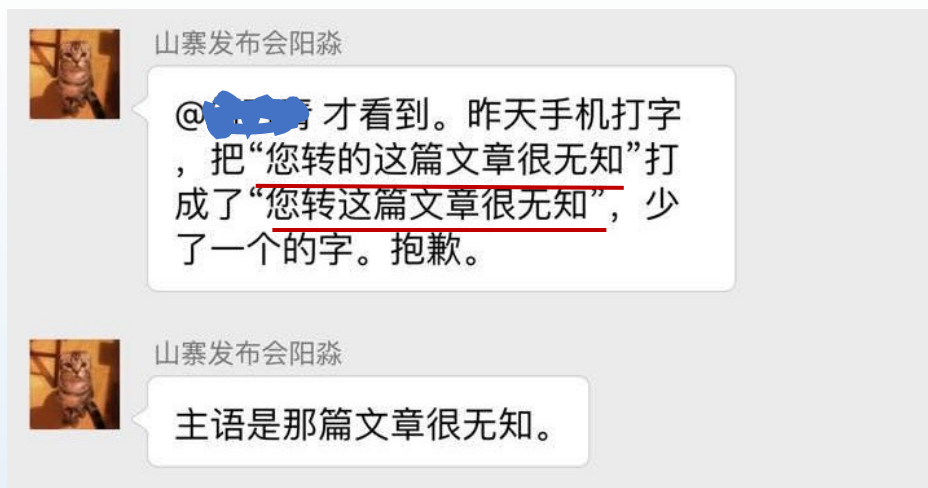
NLP任务-基础任务-中文分词



Input: sequence
Output: class for each token

NLP任务-基础任务-句法分析

- **句法分析**：分析句子的**句法成分**，如主谓宾定状补等
- **目标**：将词序列表示的句子转换成**树状结构**
- **功能**：更准确地理解句子的含义，并辅助下游NLP任务



- 只相差一个“的”字，但是表达的语义是截然不同
- 第一句话的主语是“文章”，而第二句话的主语是“转”的动作
- 进行句法分析，就可以准确地获知各自的主语，从而推导出不同的语义

NLP任务-基础任务-句法分析

□典型的句法结构表示方法:

- 短语结构句法表示

- 依存结构句法表示

□不同点: 在于依托的文法规则不一样

- 短语结构句法表示依托上下文无关文法，属于一种层次性的表示方法

- 依存结构句法表示依托依存文法

NLP任务-基础任务-句法分析

□ 短语结构句法树

□ 短语结构句法树基于**短语结构文法**（如上下文无关文法），其**核心思想**是将句子递归地划分为更小的语法单元（短语），最终形成一棵树。树的每个**内部节点**代表一个**短语**类别（如名词短语NP、动词短语VP、句子S等），**叶子节点**对应句子中的**单词**

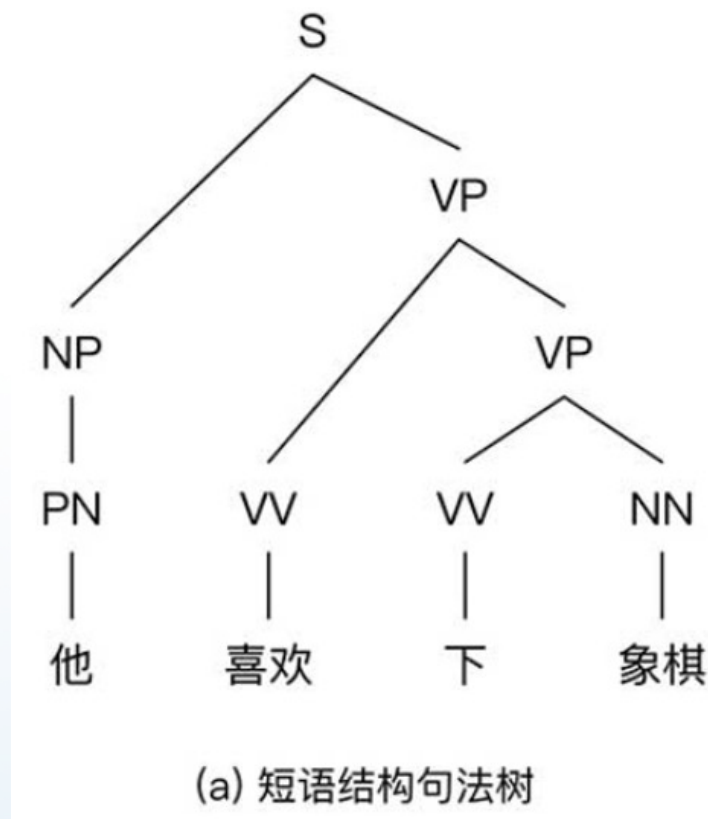
□ 结构特点

□ **层次化组合**：句子被**逐层**分解为**短语**成分，例如 “ $S \rightarrow NP VP$ ”表示一个句子由**名词短语**和**动词短语**组成

□ **非终结符**：**内部节点**用**语法标签**标记，表示该**短语的类型**

□ **终结符**：**叶子节点**直接**对应单词**（通常还带有词性标签，如NN、VB）

NLP任务-基础任务-句法分析



- S：起始符号
- NP：名词短语
- VP：动词短语

□ 短语结构句法树应用

- 短语结构树常用于需要理解句子成分层次的任务，如语法检查、机器翻译（捕捉长距离短语重排序）、语义角色标注等。

NLP任务-基础任务-句法分析

□依存结构句法树

□依存结构句法树基于依存文法，其核心思想是描述句子中单词之间的依存关系（即修饰与被修饰、支配与被支配的关系）。树中的每个节点都是一个单词，节点之间的有向弧表示依存关系，并通常附有类型标签（如主谓关系nsubj、动宾关系dobj等）。整个结构以核心谓语动词为根节点，其他词通过依存弧连接到其所依赖的词

□结构特点

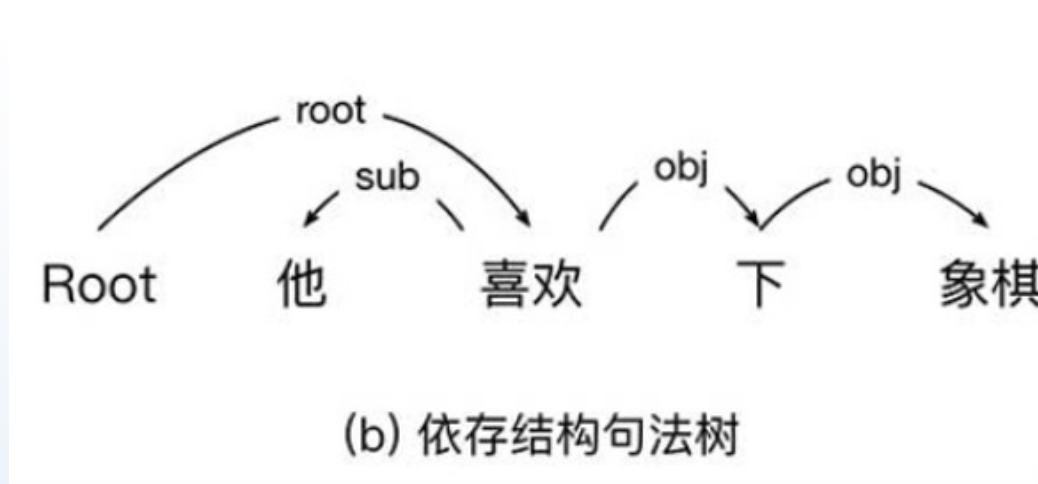
- 词间关系：直接刻画词与词之间的语法关系，不涉及短语成分
- 无短语节点：所有节点均为句子中的实际单词，没有额外的语法类别节点
- 有向弧与标签：每条弧从核心词（头）指向修饰词（从属），并标注具体关系类型

NLP任务-基础任务-句法分析

□ Sub: 主语

□ obj : 宾语

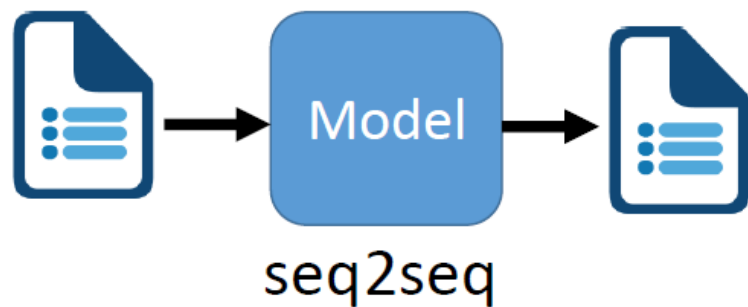
□ Root : 表示虚拟根节点，其指向整个句子的核心谓词



□ 依存结构句法树应用

- 依存句法广泛应用于信息抽取（如提取主谓宾三元组）、问答系统、情感分析等任务，因为它能直接给出词与词之间的语义关系，便于后续处理。

NLP任务-基础任务-句法分析



Input: sequence
Output: sequence

NLP任务-基础任务-语义分析

- **NLP的核心任务**: 让计算机“理解”自然语言所蕴含的意义，即语义 (Semantic)
- **语义分析**: 通过**离散的符号**及**结构**显性地表示语义
- **词义消歧**(Word Sense Disambiguation): 根据词语出现的不同上下文，确定其具体含义的NLP任务
 - **一词多义**: 例如“打”
 - “攻击” (如“打人”)
 - “玩” (如“打篮球”)
 - “编织” (如“打毛衣”)
 - **多词一义**: “马铃薯”和“土豆”
- **无统一的形式**对**句子**等语言单元的语义进行表示
 - **原因**: 语言的语义**组合性**和**进化性**，无法像词语一样使用**词典**定义**句子**、**段落**或**篇章**的语义
- **语义表示形式**分类: 根据待表示语言单元粒度以及语义表示方法的不同，可以被分为多种形式

□不同的语义表示形式，如

□语义角色标注(Semantic Role Labeling , SRL)

□语义依存分析(Semantic Dependency Parsing)

NLP任务-基础任务- 语义分析

□不同的语义表示形式，如

□语义角色标注(Semantic Role Labeling , SRL)

□也称：谓词论元结构(Predicate-Argument Structure)

□识别句子中可能的谓词(一般为动词)

□为每个谓词确定所携带的语义角色(论元)

□如表示动作发出者的施事(Agent)

□表示动作承受者的受事(Patient)

□除核心语义角色，还有一类辅助描述动作的语言成分，被称为附加语义角色，如动作发生的时间、地点和方式等

输入	他	喜欢	下	象棋	。
输出 1	施事	谓词	受事		
输出 2	施事	谓词		受事	

□两个谓词：“喜欢”和“下”

□每个谓词产生相应的论元

NLP任务-基础任务- 语义分析

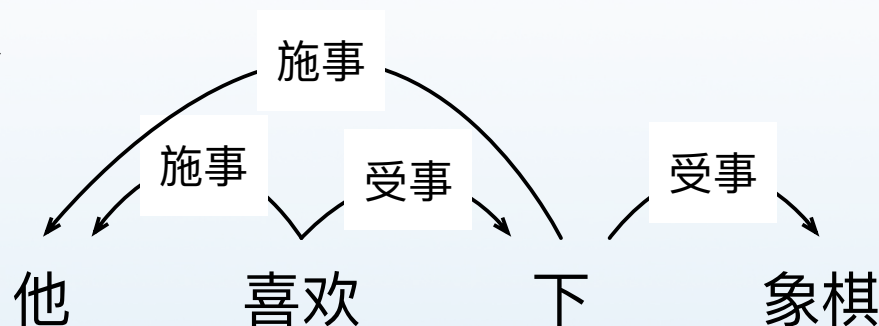
□不同的语义表示形式，如

□语义依存分析(Semantic Dependency Parsing)

□利用通用图表示更丰富的语义信息

□分类：

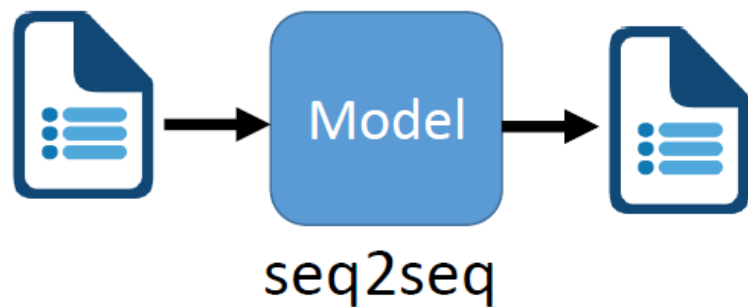
□语义依存图：图中的节点是句子中实际存在的词语、在词与词之间创建语义关系边



语义依存图分析结果示例

□概念语义图：将句子转化为虚拟的概念节点，在概念节点之间创建语义关系边

NLP任务-基础任务- 语义分析



Input: sequence
Output: sequence

目录

□ NLP任务

□ 语言模型

□ 基础任务

- 中文分词

- 字词切分

- 词性标注

- 句法分析

- 语义分析

□ 应用任务

- 信息提取

- 情感分析

- 问答系统

- 机器翻译

- 对话系统

□ 基本问题

- 文本分类

- 结构预测

- 序列到序列

NLP任务-应用任务- 信息抽取

▣信息抽取(Information Extraction , IE)

- ▣从非结构化的文本中自动提取结构化信息

 - ▣结构化的信息方便计算机进行后续的处理

 - ▣抽取的结果还可以作为新的知识加入知识库中

▣子任务:

- ▣命名实体识别(Named Entity Recognition , NER)

- ▣关系抽取(Relation Extraction)

- ▣事件抽取(Event Extraction)

NLP任务-应用任务-信息抽取

□子任务:

□命名实体识别：在文本中抽取每个提及的命名实体并标注其类型

□包括：人名、地名和机构名、专有名称(书名、电影名和药物名)

□实体链接(Entity Linking): 找到提及的命名实体后，将这些命名实体链接到知识库或知识图谱中的具体实体

□词义消歧: 如“华盛顿”

□既可以指美国首任总统

□也可以指美国首都

□需要根据上下文进行判断

□关系抽取：

□事件抽取：

NLP任务-应用任务- 信息抽取

□子任务:

- 命名实体识别：抽取每个提及的命名实体并标注其类型
- 关系抽取：识别和分类文本中提及的实体之间的语义关系,如
 - 夫妻
 - 子女
 - 工作单位和地理空间上的位置关系
 - ...二元关系

NLP任务-应用任务- 信息抽取

□子任务:

□事件抽取：从文本中识别人们感兴趣的事件以及事件所涉及的时间、地点和人物等关键元素

□事件定义：往往使用文本中提及的具体触发词(Trigger)定义

□事件抽取：与语义角色标注任务较为类似，其中

□触发词对应语义角色标注中的谓词

□事件元素则可认为是语义角色标注中的论元

□时间表达式(Temporal Expression)识别: 重要的信息抽取子任务，事件的发生时间往往比较关键

□包括两种类型的时间：绝对时间（日期、星期、月份和节假日等）和相对时间（如明天、两年前等）

□使用时间表达归一化(Temporal Expression Normalization)将这些时间表达式映射到特定的日期或一天中的时间

NLP任务-应用任务- 信息抽取

□例子：

□输入：10月28日，AMD宣布斥资350亿美元收购FPGA芯片巨头赛灵思。这两家传了多年绯闻的芯片公司终于走到了一起。

信息抽取子任务	抽取结果
命名实体识别	公司名：AMD 公司名：赛灵思
关系抽取	赛灵思 $\xrightarrow{\text{从属}}$ AMD
时间表达式抽取	10 月 28 日
时间表达式归一化	10 月 28 日 $\rightarrow$ 2020 年 10 月 28 日
事件抽取	事件：收购 时间：2020 年 10 月 28 日 收购者：AMD 被收购者：赛灵思 收购金额：350 亿美元

NLP任务-应用任务- 情感分析

□情感分析(Sentiment Analysis):使用计算机自动感知和处理人类情感

□研究人类通过文字表达的情感

□个体对外界事物的态度、观点或倾向性，如正面、负面等

□人自身的情绪 (Emotion) ，如喜怒哀惧等

□用户生成内容的情感分析，了解人们喜好，掌握舆情的发展

□如：这款手机的屏幕很不错，性能也还可以。

□NLP技术的主要应用之一

□两个主要的子任务:

□情感分类：识别文本(句子/篇章)中蕴含的情感类型或情感强度

□情感信息抽取：抽取文本中的情感元素，如评价词语、评价对象和评价搭配等

NLP任务-应用任务- 情感分析

□输入

□这款手机的屏幕很不错，性能也还可以。

情感分析子任务	分析结果
情感分类	褒义
情感信息抽取	评价词：不错；可以 评价对象：屏幕；性能 评价搭配：屏幕 ⇔ 不错；性能 ⇔ 可以

□情感分析应用场景：

□商品评论的分析、舆情分析

□工业界、社会学、经济学和管理学等领域

NLP任务-应用任务- 问答系统

□ 问答系统 (Question Answering , QA)

- 用户以自然语言形式描述问题，从异构数据中获得答案

□ 根据数据源的不同，问答系统可以分为4种主要的类型

□ 检索式问答系统

- 答案来源于固定的文本语料库或互联网，系统通过查找相关文档并抽取答案完成问答

□ 知识库问答系统

- 回答问题所需的知识以数据库等结构化形式存储，问答系统首先将问题解析为结构化的查询语句，通过查询相关知识点，并结合知识推理获取答案

□ 常问问题集问答系统

- 通过对历史积累的常问问题集合进行检索，回答用户提出的类似问题

□ 阅读理解式问答系统

- 通过抽取给定文档中的文本片段或生成一段答案来回答用户提出的问题

NLP任务-应用任务- 机器翻译

□机器翻译(Machine Translation , MT):

- 利用计算机实现从一种自然语言(源语言)到另外一种自然语言(目标语言)的自动翻译
- 机器翻译一直是NLP领域主要的研究任务和应用场景
- 最终目标：任意时间、任意地点和任意语言之间的自动翻译
- 翻译服务：
 - 在线翻译：谷歌、百度
 - 翻译机：科大讯飞将语言的语音翻译为另一种语言的语音

NLP任务-应用任务-机器翻译

□ 机器翻译(Machine Translation , MT):

□ 例子：

S:	北京 是 中国 的 首都 。
T:	Beijing is the capital of China .

□ 方法分类：机器翻译方法一般以句子为基本输入单位，研究从源语言句子到目标语言句子的映射函数

□ 理性主义：基于规则

□ 经验主义：数据驱动的统计方法，在机器翻译领域表现为基于语料库(翻译实例库)的研究方法

□ 神经机器翻译：深度学习(利用深度神经网络学习源语言句子到目标语言句子的隐式翻译规则)

NLP任务-应用任务-对话系统

□对话系统(Dialogue System): 以自然语言为载体，用户与计算机通过**多轮交互**的方式实现特定目标的智能系统

□**特定目标**: 完成特定任务、获取信息或推荐、获得情感抚慰和社交陪伴

□**应用场景**：语音助手、智能音箱和车载语音系统

□**典型示例**：20世纪50年代，图灵提出用于评测计算机系统智能化水平的“图灵测试”

□**主要分类**：

□**任务型**对话系统(Task-Oriented Dialogue)

□**开放域**对话系统(Open-Domain Dialogue)

NLP任务-应用任务- 对话系统

□主要分类：

□开放域对话系统：

- 目标: 以社交为目标的对话系统，如：闲聊、情感陪护
- 特点: 被称为聊天系统或聊天机器人(Chatbot), 在领域和话题上具有很强的开放性

□示例：

U: 今天天气真不错!
S: 是啊, 非常适合室外运动。
U: 你喜欢什么运动?
S: 我喜欢踢足球, 你呢?

- 算法主要目标：提升对话的轮次以及用户的满意度。相比对话的准确性，更关注对话的多样性以及对用户的吸引程度

NLP任务-应用任务-对话系统

□主要分类：

□任务型对话系统：

□特点：任务导向型的对话系统

□主要应用：垂直领域的自动业务助理等，具有明确的任务目标

□如完成机票预订、天气查询等特定的任务

□三个模块构成：

□自然语言理解：分析用户话语的语义，通常的表示形式为该话语的领域、意图以及相应的槽值等

□对话管理：

□自然语言生成：

NLP任务-应用任务-对话系统

□任务型对话系统：

□三个模块构成:

□自然语言理解: 分析用户话语的语义，通常的表示形式为该话语的领域、意图以及相应的槽值等

□示例：

U: 帮我订一张明天去北京的机票

NLU 子任务	分析结果
领域	机票
意图	订机票
槽值	出发时间 = 明天；到达地 = 北京；数量 = 一张

□对话管理:

□自然语言生成:

NLP任务-应用任务-对话系统

任务型对话系统：

自然语言理解：分析用户话语的语义及相应的槽值等

对话管理

对话状态跟踪

对话状态：语义槽和值的列表

对话策略优化

选择下一步采用什么样的策略

自然语言生成：写模板即可实现

U：帮我订一张明天去北京的机票

NLU 子任务	分析结果
领域	机票
意图	订机票
槽值	出发时间 = 明天；到达地 = 北京；数量 = 一张

对话状态跟踪

[到达地=北京；出发时间=明天；出发地=NULL；数量=1]

策略优化

- 询问出发地
- 询问舱位类型

请问您从哪里出发？

- 语音合成

用户

三个模块一直循环执行，随着每次用户的话语不同，对话状态也随之变化。采用不同的回复策略，直到满足用户的订票需求为止。

NLP任务-应用任务-对话系统

□对话系统 (Dialogue System)

	任务型 Task	聊天 Chat	知识问答 Knowledge	推荐 Recommendation
目的	完成任务或动作	闲聊	知识获取	信息推荐
领域	特定域 (垂类)	开放域	开放域	特定域
以话轮数评价	越少越好	话轮越多越好	越少越好	越少越好
应用	虚拟个人助理	娱乐、情感陪护	客服、教育	个性化推荐
典型系统	Siri、Cortana、Google Assistant、度秘	小冰、笨笨、chatGPT	Watson、Wolfram Alpha	阿里小蜜

目 录

CONTENTS

语言模型

基础任务

应用任务

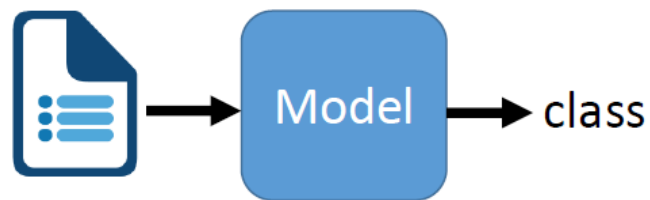
基本问题

- 文本分类问题
- 结构预测问题
- 序列到序列问题

基本问题-文本分类问题

- 将输入文本映射为所属类别(预定义的**封闭集合**)
- 最简单**最基本**的自然语言处理问题
- 应用场景
 - **垃圾邮件过滤**: 垃圾、非垃圾
 - **新闻分类**: 政治、经济和体育等
 - **情感分类**: 两类(褒、贬)、多类(喜、怒、哀和惧等)
- 很多问题可以**转化**为文本分类问题
 - **文本匹配**(Text Matching): 判断两段输入文本之间的匹配关系
 - **复述**关系(Paraphrasing): 判断两个表述不同的文本语义是否相同
 - **蕴含**关系(Entailment): 根据一个前提文本, 推断与假设文本之间的蕴含或矛盾关系
 - **一种转换**方法: 是将两段文本直接拼接起来, 按复述或非复述、蕴含或矛盾等关系分类

基本问题- 文本分类问题



输出概率



分类器



文本表示

文本表示计算 (编码器)



我



喜欢



打



篮球

词表示

机器学习分类的示意图

基本问题-文本分类

文本匹配问题

□判断两段文本之间的匹配关系

- 如复述关系、蕴含关系等

□解决方案

□双塔结构

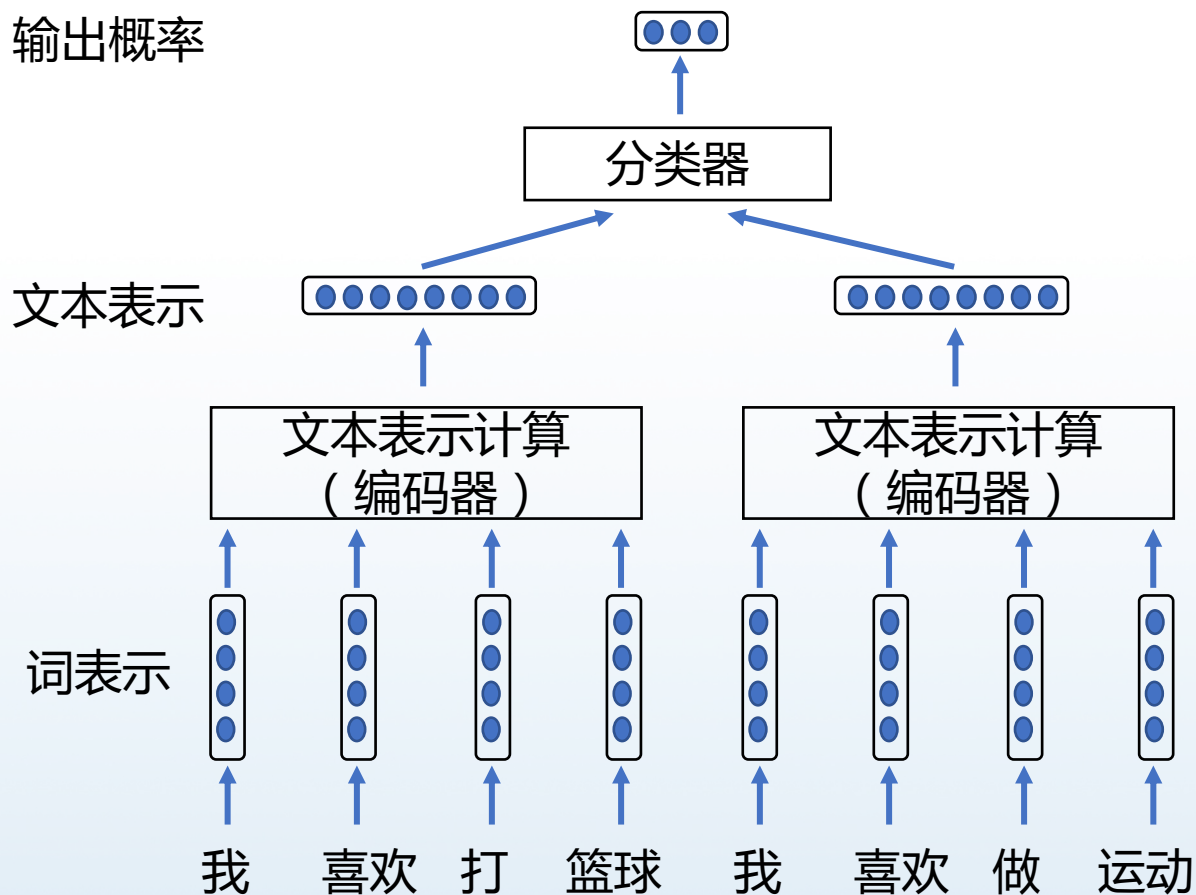
- 两段文本分别通过两个模型映射为向量，然后判断两个向量之间的匹配关系

□单塔结构

- 将两段文本直接拼接，然后进行匹配关系分类

基本问题-文本分类问题

文本匹配



双塔结构示意图

基本问题-结构预测问题

□ 输出类别之间具有较强的相互关联性

□ 自然语言处理的本质问题

□ 三种典型的结构预测问题

□ 序列标注

□ 为输入文本序列中的每个词标注相应的标签

□ 输入词和输出标签数目相同且一一对应

□ 如词性标注：

输入	他	喜欢	下	象棋	。
输出	PN	VV	VV	NN	PU

□ 序列分割

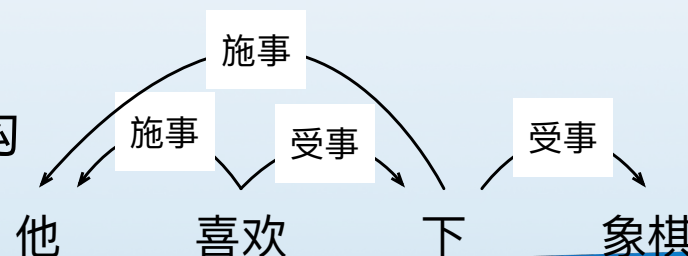
□ 在文本序列中切分出子序列

□ 可以转化为序列标注问题

输入	我	爱	北	京	天	安	门	。
分词输出	B	B	B	I	B	I	I	B
命名实体输出	O	O	B-LOC	I-LOC	I-LOC	I-LOC	I-LOC	O

□ 图结构生成

□ 输入自然语言，输出以图表示的结构

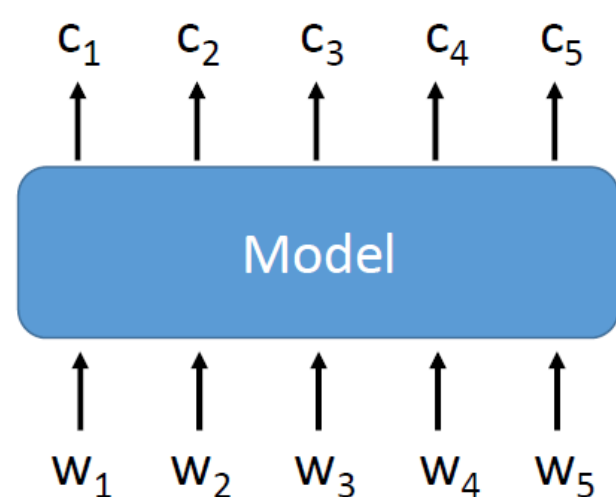


基本问题-结构预测问题

序列标注

□任务示例：词性标注

□方法：



□文本分类：简单地看成多个独立的文本分类问题，即针对每个词提取特征，然后进行标签分类，并不考虑输出标签之间的关系

□条件随机场(Conditional Random Field, CRF)

□不但考虑了每个词属于某一标签的概率(发射概率)，还考虑了标签之间的相互关系(转移概率)

□循环神经网络模型：隐含地建模了标签之间的相互关系

□为了进一步提高准确率，也可以在循环神经网络之上再使用条件随机场模型

基本问题-结构预测问题：序列分割

- 序列分割：在文本序列中切分出子序列
- 任务示例：分词、字词切分、命名实体识别
- 求解：

- 转化为序列标注问题
- 专门序列分割模型

分词结果：我 爱 北京 天安门。

输入	我	爱	北	京	天	安	门	。
分词输出	B	B	B	I	B	I	I	B
命名实体识别输出	O	O	B-LOC	I-LOC	I-LOC	I-LOC	I-LOC	O

□ 分词：

□ B：该字符是一个词的开始

□ I：该字符是一个词中间

命名实体识别

□ B-XXX：实体的开始

□ I：中间

□ I-XXX：中间

□ O：代表其他

□ O：非实体

□ XXX：实体的类型，如人名（PER）、地名（LOC）和机构名（ORG）

□ B：开始

命名实体识别结果：北京天安门=LOC

基本问题-结构化预测问题：图结构生成

□图结构生成

- 输入自然语言，输出以图表示的结构

□任务：

□句法分析

- 依存结构句法分析
- 短语结构句法分析

□语义分析

- 语义角色标注
- 语义依存分析

□图结构生成算法

- 基于图的算法
- 基于转移的算法

基本问题-结构化预测问题：图结构生成

□基于图的算法

- 输入的词为节点

- 两个节点构成的边赋予一定的分数

- 根据解码算法求解出一个满足约束的分数最大的子图

- 关键问题

 - 解码算法

 - 如何为边打分以

 - 参数如何优化

基本问题-结构化预测问题：图结构生成

基于转移的算法

□ 将图结构的构建过程转化为状态转移序列

□ 如,标准弧转移算法

□ 转移状态由一个栈和一个队列构成

□ 三种转移动作

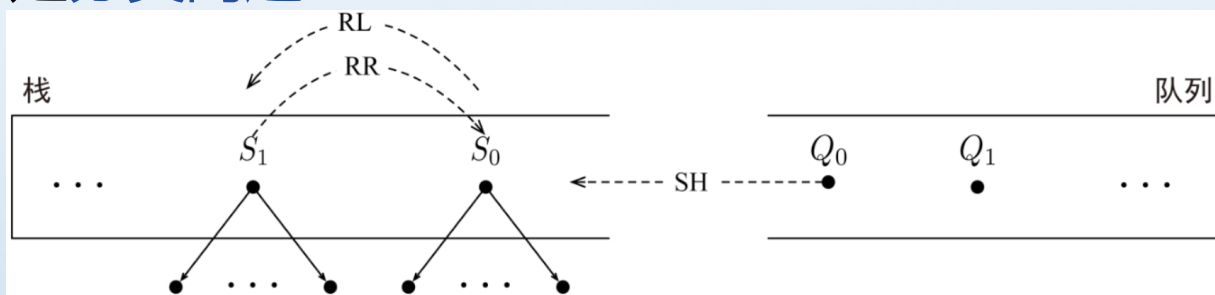
□ 移进(SH): 队列中的第一个元素移入栈顶, 形成一个仅包含一个节点的依存子树

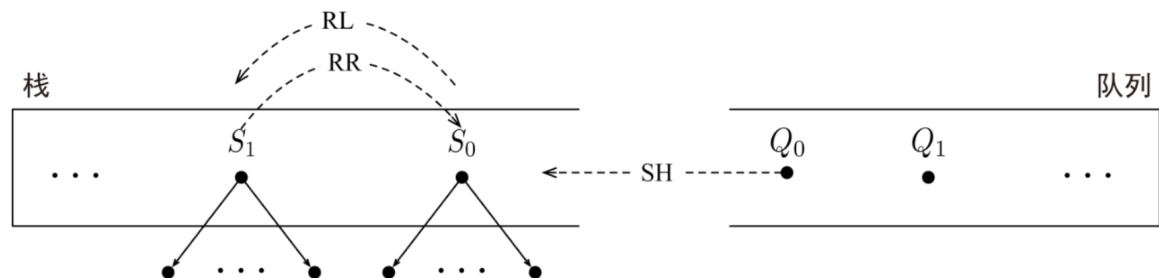
□ 左弧(RL): 将栈顶的两棵依存子树采用一个左弧 $S_1 \curvearrowright S_0$ 进行合并, S_1 下栈

□ 右弧(RR): 将栈顶的两棵依存子树采用一个右弧 $S_1 \curvearrowleft S_0$ 进行合并, S_0 下栈

□ 对旧状态执行一个转移动作转换为新状态

□ 转移动作的选择本质上也是分类问题





步骤	栈	队列	下一步动作
0		他 喜欢 下 象棋	SH
1		他 喜欢 下 象棋	SH
2	他 喜欢	下 象棋	RL
3	喜欢	下 象棋	SH
	他		
4	喜欢 下 象棋		SH
	他		
5	喜欢 下 象棋		RR
	他		
6	喜欢 下		RR
	他 象棋		
7	喜欢		FIN
	他 下 象棋		

基本问题-序列到序列

□ 将输入序列转换为输出序列

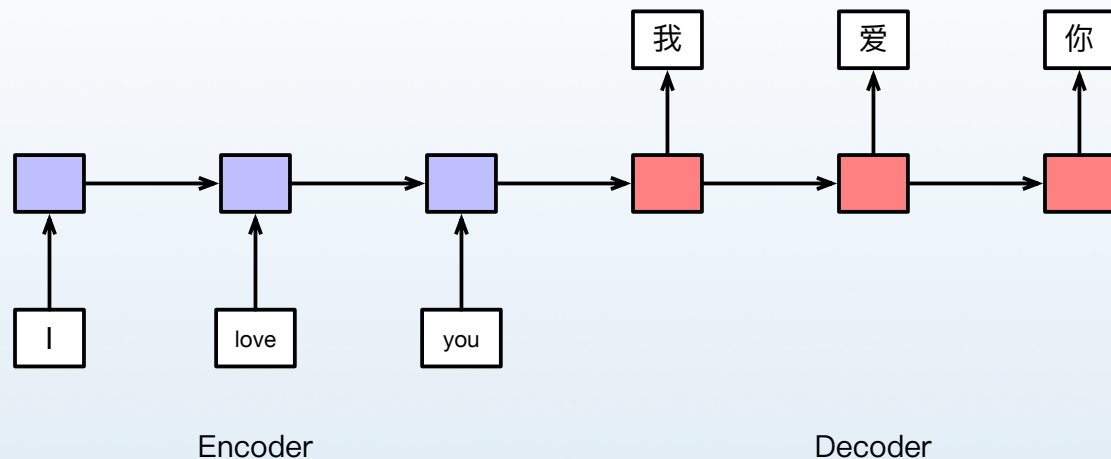
□ 输入和输出的序列 **不要求等长**，也 **不要求词表一致**

□ 泛化为“编码器—解码器”模型（Encoder-Decoder）

□ 本质上也是 **分类问题**

□ 典型任务

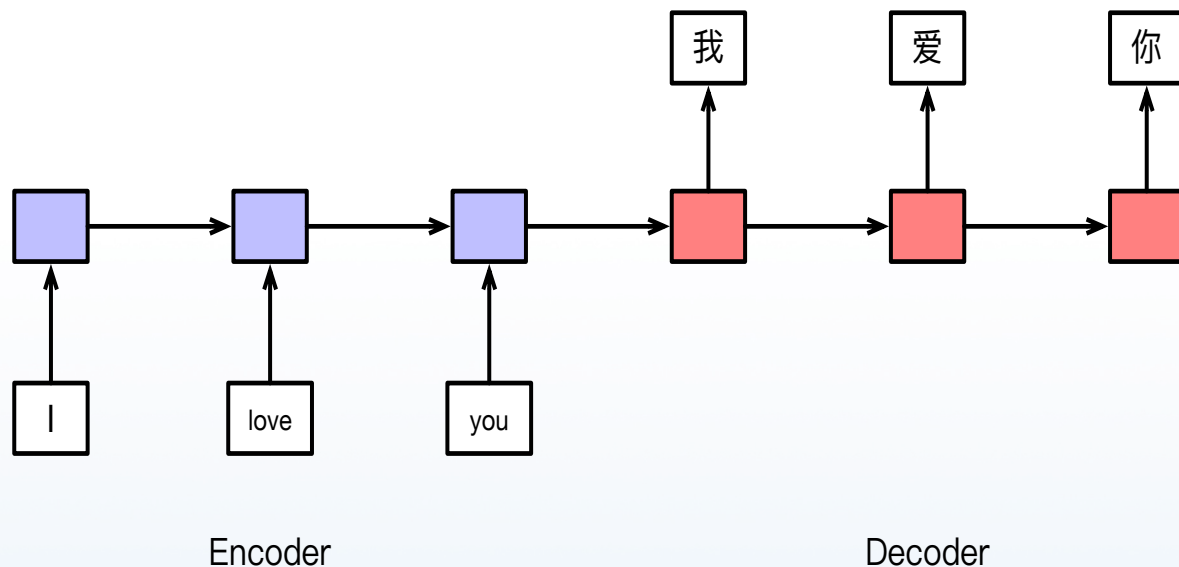
任务	输入	输出
机器翻译	源语言	目标语言
文本摘要	原文	摘要
回复生成	用户语句	机器回复
图片描述生成	图片	文本描述
语音识别	语音	转换文本



“编码器—解码器”模型示例

基本问题-序列到序列

深度学习 “编码器—解码器” 模型(Encoder-Decoder)



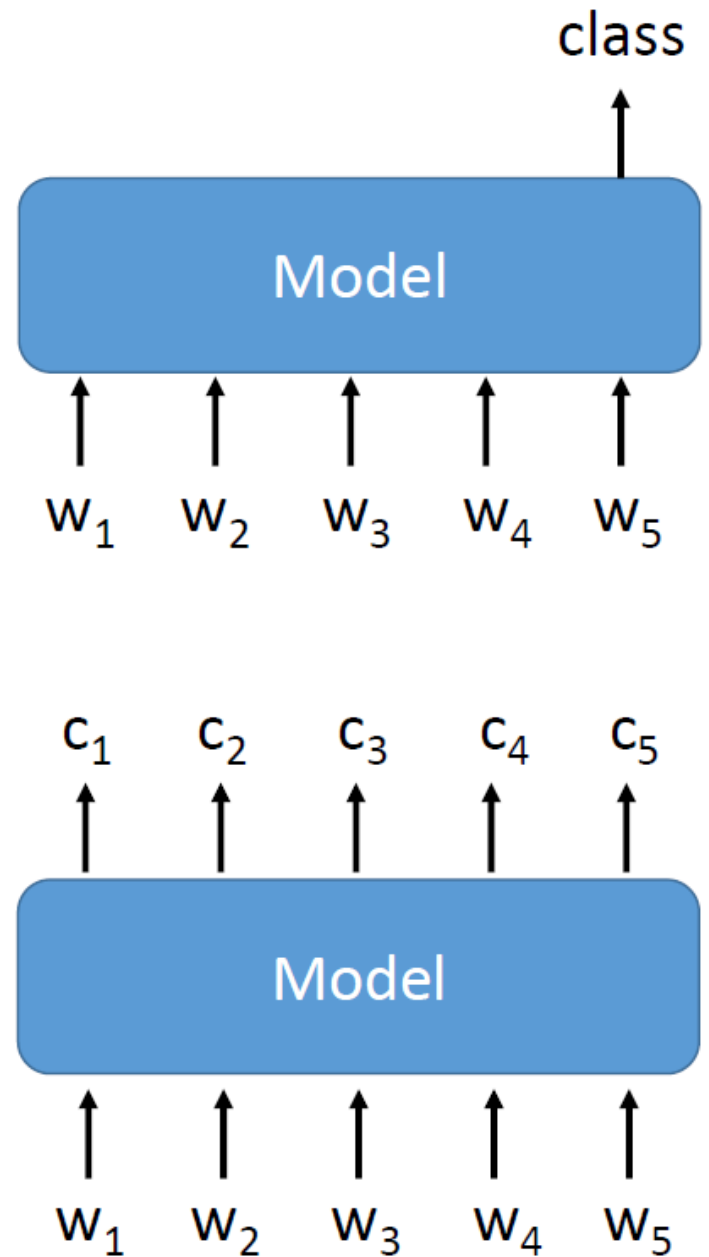
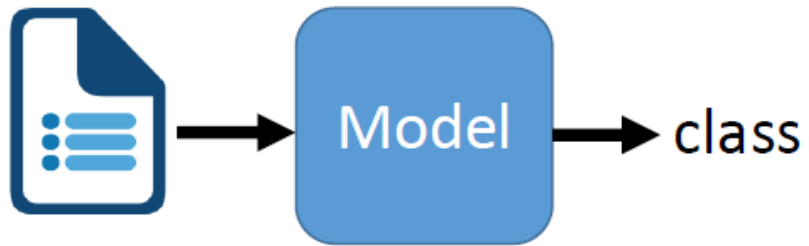
“编码器—解码器” 模型示例

- ❑ 编码器(En-coder): 直接将输入序列表示为一个向量
- ❑ 解码器(Encoder-Decoder): 向量生成输出序列
- ❑ 序列到序列模型具备强大的建模能力，其已成为自然语言处理的大一统框架

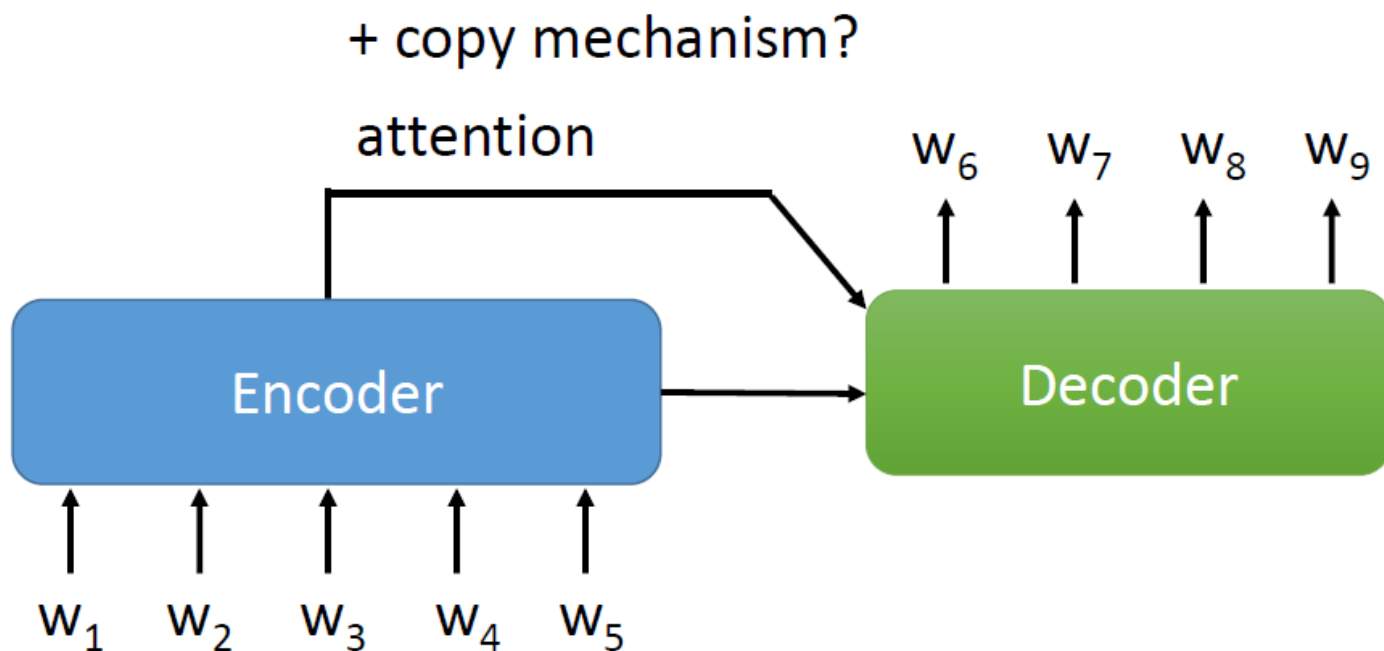
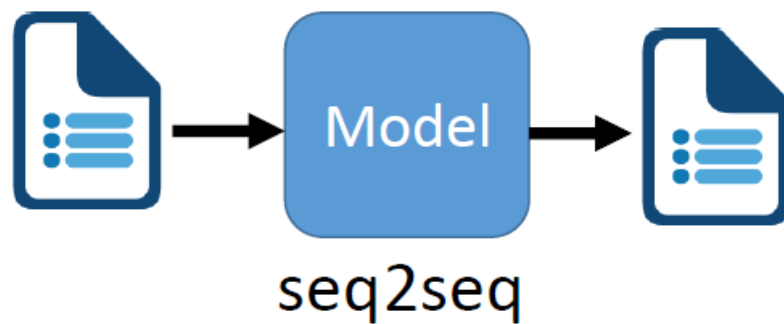
基本问题

- 文本分类问题
- 结构预测问题
- 序列到序列问题

Category



Category

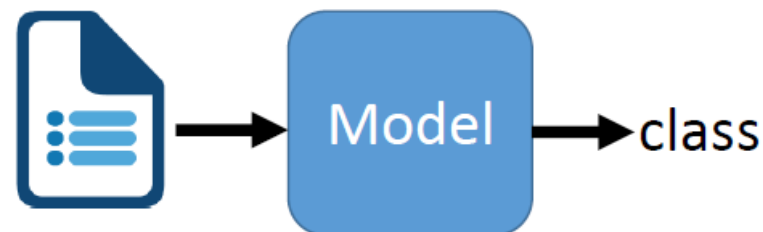
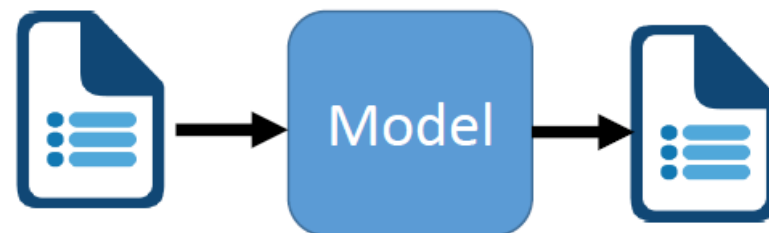
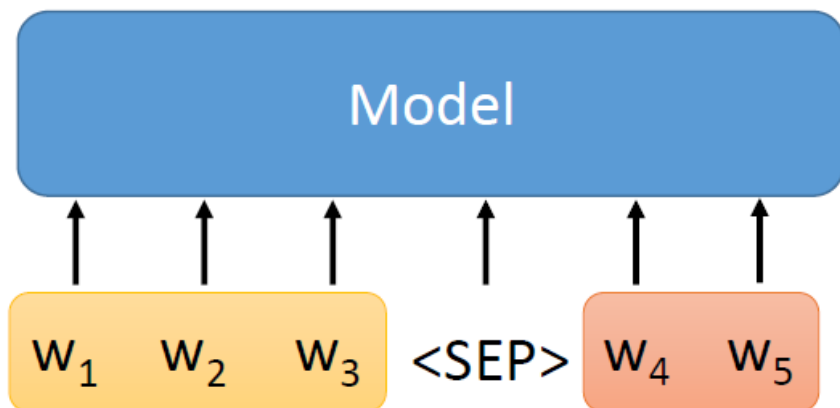


Category

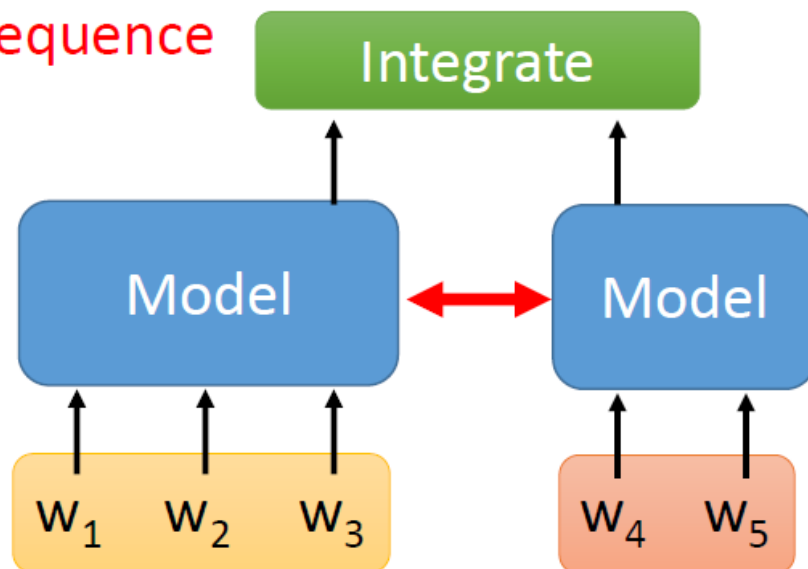
What happens if there are more than two input sequences?



Simply concatenate ...



Attention between
sequence



自然语言处理的评价

- 由于自然语言处理任务的多样性以及评价的主观性，很难使用单一的评价指标衡量所有任务的性能，所以针对不同类型的任务，往往采用不同的评价方法。
- 对评价方法的准确把握，有助于深入理解各项自然语言处理任务。

自然语言处理的评价

□ 准确率(Accuracy)

- 最简单直观的评价指标，常被用于文本分类、词性标注等问题

$$ACC^{cls} = \frac{\text{正确分类的文本数}}{\text{测试文本总数}}$$

$$ACC^{pos} = \frac{\text{正确标注的词数}}{\text{测试文本中词的总数}}$$

- 并非全部的序列标注问题都可以采用准确率进行评价

- 例如：将分词、命名实体识别等序列分割问题转化为序列标注问题后，就不应该使用准确率进行评价。

- 例：命名实体识别，如果采用按词计算的准确率，

- 非命名实体也被计入准确率的计算之中

- 如果错标了部分词，那么命名实体识别结果就是错误的，但是按照词准确率计算的话，仍然有部分词被认为分类正确了

自然语言处理的评价

□ 准确率(Accuracy)

□ 命名实体转化为序列标注问题

输入	张	三	是	哈	尔	滨	人	。
正确标注序列	B-PER	I-PER	O	B-LOC	I-LOC	I-LOC	O	O
预测标注序列	B-PER	O	O	B-LOC	I-LOC	I-LOC	O	O

□ 准确率： $7/8=0.875$

□ 实际：

□ 测试文本中命名实体的总数：2（“张三”和“哈尔滨”）

□ 识别出的命名实体总数：2（“张”和“哈尔滨”）

□ 正确识别的命名实体数目：1（“哈尔滨”）

自然语言处理的评价

□ F值

□ 针对某一类别的评价

$$F \text{ 值} = \frac{(\beta^2 + 1)PR}{\beta^2(P + R)}$$

$$P = \frac{\text{正确识别的命名实体数目}}{\text{识别出的命名实体总数}}$$

$$R = \frac{\text{正确识别的命名实体数目}}{\text{测试文本中命名实体的总数}}$$

□ β 是加权调和参数； P 是精确率(Precision)； R 是召回率(Recall)

□ 当权重为 $\beta = 1$ 时，表示精确率和召回率同样重要，也称F1值

自然语言处理的评价

□ 准确率(Accuracy)

□ 命名实体转化为序列标注问题

输入	张	三	是	哈	尔	滨	人	。
正确标注序列	B-PER	I-PER	O	B-LOC	I-LOC	I-LOC	O	O
预测标注序列	B-PER	O	O	B-LOC	I-LOC	I-LOC	O	O

□ 准确率： $7/8=0.875$

□ 实际：

□ 实际命名实体的总数：2（“张三”和“哈尔滨”）

□ 识别出的命名实体总数：2（“张”和“哈尔滨”）

□ 正确识别的命名实体数目：1（“哈尔滨”）

□ 精确率： $p=1/2=0.5$

□ 召回率： $R=1/2=0.5$

□ $F_1=0.5$

自然语言处理的评价

□ 依存分析的评价

□ UAS (Unlabeled Attachment Score)

- 词的父节点被正确识别的准确率

□ LAS (Labeled Attachment Score)

- 词的父节点以及与父节点的句法关系都被正确识别的准确率

□ 机器翻译的评价

□ BLEU (BiLingual Evaluation Understudy) 值

- 统计机器译文与参考译文 (可以不只有一个) 中N-gram匹配的数目占机器译文中所有N-gram总数的比率

□ 对话系统的评价

- 由于回复的开放性，没有标准答案，很难自动评价
- 由于对话的交互性，不能简单地通过一轮人机对话就对系统进行评价
- 目前往往采用人工评价 (流畅度、相关度、准确性等等)



Thanks!
Q&A ?

wuxianyan@njau.edu.cn